

Week2 实验报告

班级： 25 级计算机科学与技术

学号： 25020007028

姓名： 官靓颖

任课老师： 周小伟

日期： 2026 年 8 月 31 日

第一部分：实验习题

第一题：控制一个可清理的后台任务

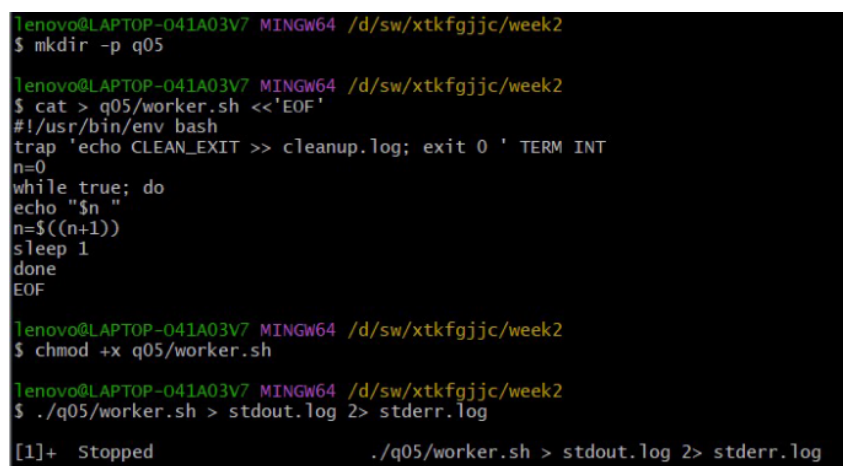
主题：命令行环境：进程、信号与任务控制

将下方脚本保存为 q05/worker.sh，再完成任务控制

给定内容

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0 ' TERM INT
n=0
while true; do
echo "$n "
n=$((n+1))
sleep 1
done
```

1. 赋予脚本执行权限，在前台启动，并把 stdout、stderr 分别写入 stdout.log、stderr.log



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ mkdir -p q05

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat > q05/worker.sh <<'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0 ' TERM INT
n=0
while true; do
echo "$n "
n=$((n+1))
sleep 1
done
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ chmod +x q05/worker.sh

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ ./q05/worker.sh > stdout.log 2> stderr.log

[1]+  Stopped                  ./q05/worker.sh > stdout.log 2> stderr.log
```

图 1: 0101

知识点：

1. `chmod +x`：为脚本添加可执行权限
2. `>`：重定向标准输出 `stdout` 到文件
3. `2>`：重定向标准错误 `stderr` 到文件
4. `trap` 捕获 `SIGINT`(Ctrl-C)、`SIGTERM` 信号，收到信号执行清理逻辑写入 `cleanup.log`

2. 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续；使用 jobs 确认状态

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ bg
[1]+ ./q05/worker.sh > stdout.log 2> stderr.log &

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ jobs
[1]+  Running                  ./q05/worker.sh > stdout.log 2> stderr.log &
```

图 2: 0102

知识点:

1. Ctrl+Z 发送 SIGTSTP 信号，挂起（暂停）前台作业；bg 将挂起作业放到后台继续运行。
2. jobs 查看 shell 会话内作业列表，显示作业编号、运行状态与命令；作业是 shell 层面对任务的管理。
3. 使用 jobs -p 或等价方式取得 PID，不得手工抄写 PID；发送 SIGTERM 并等待任务结束

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ kill $(jobs -p)
[1]+  Done                    ./q05/worker.sh > stdout.log 2> stderr.log

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ wait $(jobs -p)
```

图 3: 0103

知识点:

1. jobs -p 获取后台作业对应的 PID；\$(command) 命令替换，把命令输出作为参数传入。
2. kill 默认发送 SIGTERM(15)，允许程序捕获信号执行清理；wait 等待进程结束。
3. kill -9(SIGKILL) 无法被 trap 捕获，会直接强制销毁进程，清理逻辑不会执行，禁止使用。
4. 确认 cleanup.log 中出现 CLEAN_EXIT，且任务已经退出；禁止使用 kill -9

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat cleanup.log
CLEAN_EXIT

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ jobs

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ |
```

图 4: 0104

知识点:

1. cat 读取文件内容，验证 trap 清理逻辑正常执行；jobs 无输出代表作业已经结束。
2. SIGTERM、SIGINT 可以被 trap 捕获实现优雅退出；SIGKILL 信号不可捕获，无法执行清理。

第二题：语义重构与本地开发反馈

主题：开发环境与工具

在 q06 中按照给定代码创建三个 Python 文件，并使用编辑器语言服务器完成一次语义重构

给定内容

```
# math_utils.py
def total_price(price:float , count : int) -> float :
    return price * count
# app.py
from math_utils import total_price
print(total_price(12 .5 , 4))
# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12 .5 , 4) == 50 .0
```

1. 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器

输入：

```
mkdir -p q06
cat > q06/math_utils.py <<'EOF'
def total_price(price:float , count : int) -> float :
    return price * count
EOF
cat > q06/app.py <<'EOF'
from math_utils import total_price
print(total_price(12.5 , 4))
EOF
cat > q06/test_math_utils.py <<'EOF'
from math_utils import total_price
def test_total_price():
    assert total_price(12.5 , 4) == 50.0
EOF
```

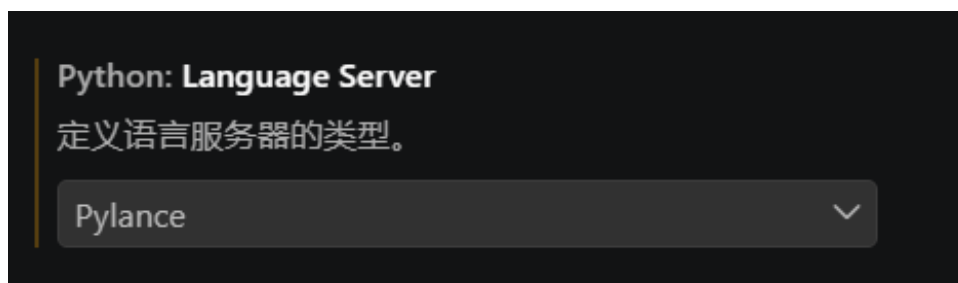


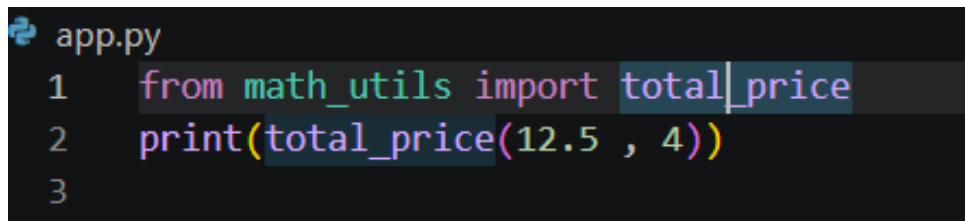
图 5: Python 语言服务器

知识点：

1. Python 语言服务器 (Pylance) 提供静态语义分析：类型提示、代码跳转、重构能力；区别于单纯语法高亮。
2. `pytest` 是 Python 单元测试框架；`ruff` 是高性能 linter(代码检查工具)，可检测未使用导入、格式问题。

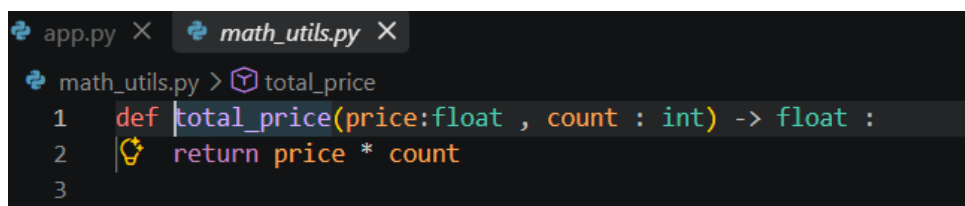
2. 分别演示跳转到定义和查找引用

- 跳转到定义：在 `app.py` 中光标放在 `total_price`，按下 F12，跳转到 `math_utils.py` 函数定义处
- 查找引用：光标放在 `total_price`，按下 Shift+F12，列出所有引用该函数的位置



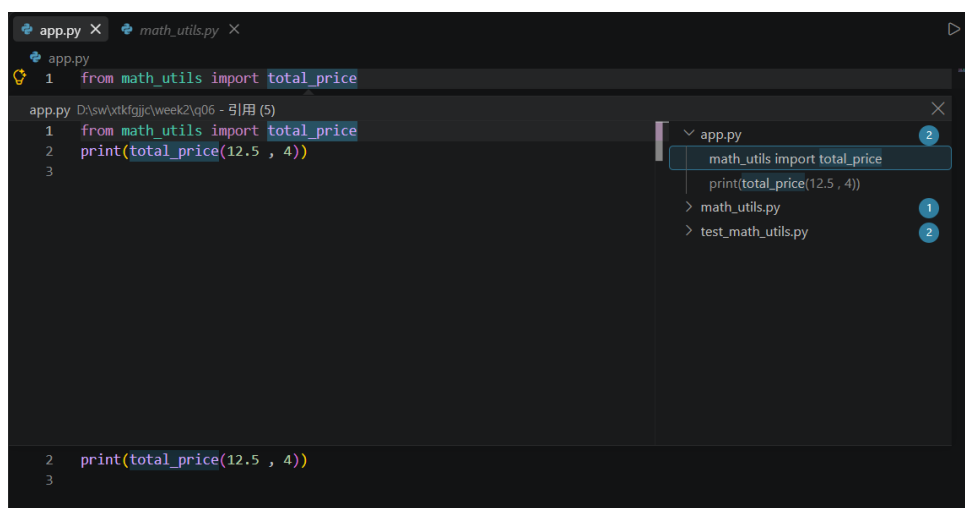
```
app.py
1  from math_utils import total_price
2  print(total_price(12.5 , 4))
3
```

图 6: app.py



```
app.py ×  math_utils.py ×
math_utils.py > total_price
1  def total_price(price:float , count : int) -> float :
2  |  return price * count
3
```

图 7: 函数定义处



```
app.py ×  math_utils.py ×
app.py
1  from math_utils import total_price
2  print(total_price(12.5 , 4))
3

app.py D:\sw\python\python311\python.exe - 引用 (5)
1  from math_utils import total_price
2  print(total_price(12.5 , 4))
3

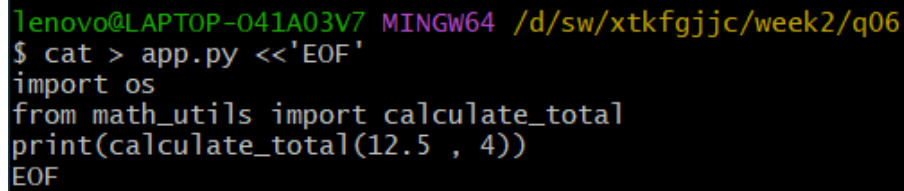
2  print(total_price(12.5 , 4))
3

app.py
  math_utils import total_price
  | print(total_price(12.5 , 4))
  > math_utils.py
  > test_math_utils.py
```

图 8: 所有引用该函数处

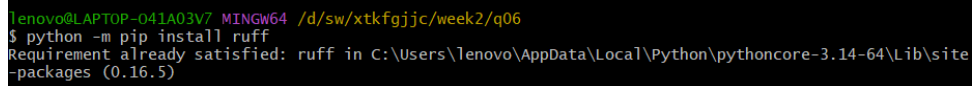
知识点：

2. 对比普通文本替换：文本替换会误改字符串、注释中同名文字；语义重命名只修改程序符号。
3. 修改会同时更新定义、导入语句、函数调用，跨文件同步变更。
4. 在 `app.py` 中临时加入未使用的 `import os`，用 `ruff` 发现并删除；最后运行 `ruff` 和 `pytest` 并保证通过



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q06
$ cat > app.py <<'EOF'
import os
from math_utils import calculate_total
print(calculate_total(12.5 , 4))
EOF
```

图 12: 加入



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q06
$ python -m pip install ruff
Requirement already satisfied: ruff in C:\Users\lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (0.16.5)
```

图 13: ruff

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q06
$ python -m ruff check
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
1 | / import os
2 | | from math_utils import calculate_total
| | _____^
3 | | print(calculate_total(12.5 , 4))
|
help: Organize imports
|
1 | import os
2 | +
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5 , 4))
|

F401 [*] `os` imported but unused
--> app.py:1:8
|
1 | import os
| | ^^^
2 | from math_utils import calculate_total
3 | print(calculate_total(12.5 , 4))
|
help: Remove unused import: `os`
|
- import os
1 | from math_utils import calculate_total
|

Found 2 errors.
[*] 2 fixable with the `--fix` option.

```

图 14: ruff check

```

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q06
$ python -m ruff check --fix
Found 2 errors (2 fixed, 0 remaining).

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q06
$ python -m pytest test_math_utils.py -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\lenovo\AppData\Local\Python\pyt
honcore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: D:\sw\xtkfgjjc\week2\q06
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.01s =====

```

图 15: 通过

知识点:

1. ruff 代码检查工具，规则 F401 代表：导入模块但是没有被使用。--fix 可以自动修复部分代码问题。
2. pytest 运行单元测试，断言通过代表代码逻辑没有因为重构发生错误。

3. 静态工具 (ruff) 做代码质量检查；单元测试 (pytest) 做逻辑正确性验证，二者配合保障开发质量。

第三题：用调试器定位归并排序缺陷

主题：Debugging

将下列存在缺陷的归并排序代码保存为 `q07/merge_sort.py`，再使用调试器定位并修复

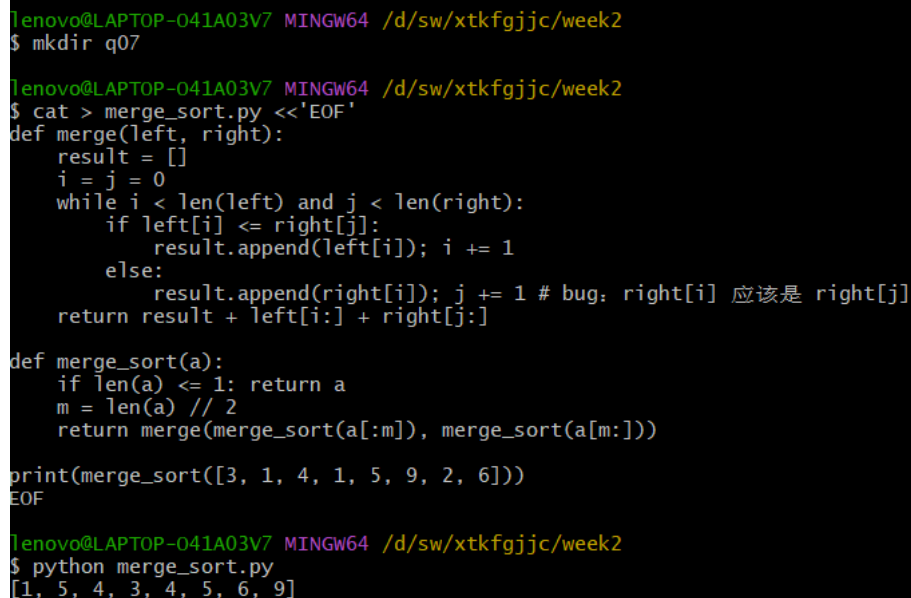
给定内容

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

1. 先运行给定输入，确认结果错误或出现异常



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ mkdir q07

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # bug: right[i] 应该是 right[j]
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 16: 输出错误

知识点：

1. 错误原因: else 分支中, 取的是 `right[i]`, 应该是 `right[j]`, 下标变量写错, 会触发列表索引越界异常。
2. 使用 `pdb`、IDE debugger 或等价调试器, 在 `merge` 中设置断点并观察 `i`、`j`、`left[i]`、`right[j]`

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            import pdb; pdb.set_trace() # 断点在这里
            result.append(right[i]); j += 1 # 错误行
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF
```

图 17: 设置断点

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ python merge_sort.py
> d:\sw\xtkfgjjc\week2\merge_sort.py(8)merge()
-> import pdb; pdb.set_trace() # 断点在这里
(Pdb) p i
0
(Pdb) p j
0
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p right[i]
1
(Pdb) q
Quitting pdb will kill the process. Quit anyway? [y/n] y
```

图 18: 观察输出

知识点:

1. 调试器可以查看运行时变量的值, 定位下标变量混用的逻辑错误;
2. 循环内两个数组分别使用独立指针 `i`、`j`, 不能混淆。

3. 完成最小修复，不得用 sorted 替换归并排序

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat > merge_sort.py <<'EOF'
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 修复: right[i] -> right[j]
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 19: 修复

知识点:

1. 最小修复只修改错误下标: `right[i]` 修改为 `right[j]`;
2. 归并排序核心逻辑: 分治, 递归拆分, 合并两个有序子数组。

4. 使用 pytest 增加两个测试: 给定输入和含重复元素的列表; 保证测试通过

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat > test_merge_sort.py <<'EOF'
import pytest
from merge_sort import merge_sort

def test_original_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    assert merge_sort([2, 2, 1, 3, 2, 1]) == [1, 1, 2, 2, 2, 3]

if __name__ == "__main__":
    pytest.main()
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ python -m pytest test_merge_sort.py -v

test session starts
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\lenovo\AppData\Local\Python\pyt
honcore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: D:\sw\xtkfgjjc\week2
collected 2 items

test_merge_sort.py::test_original_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.03s =====
```

图 20: 测试

知识点:

1. pytest 用于自动化单元测试, 验证排序函数正确性;
2. 测试用例需要覆盖普通序列、包含重复元素的序列, 验证归并排序稳定性。

第四题：先测量，再优化慢速词频程序

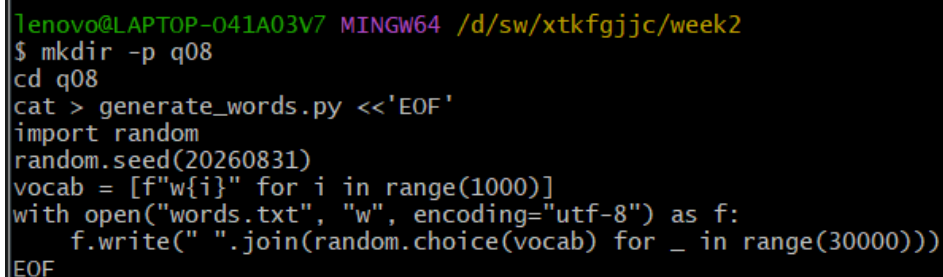
主题：Profiling

在 q08 中创建可重复生成的词表和故意低效的去重程序，再使用性能分析工具优化

给定内容

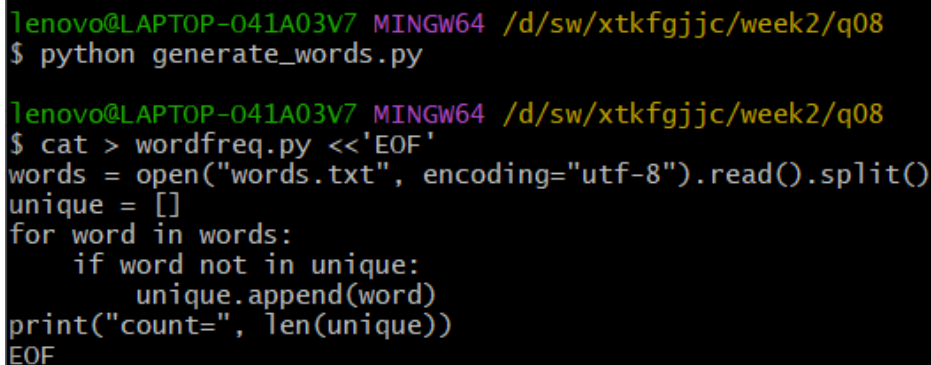
```
# generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

1. 生成 words.txt，使用 time 运行原程序 2 次并记录耗时



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ mkdir -p q08
cd q08
cat > generate_words.py <<'EOF'
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF
```

图 21: generate_word.py



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ python generate_words.py

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ cat > wordfreq.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
```

图 22: wordfreq.py

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ time python wordfreq.py
count= 1000

real    0m0.426s
user    0m0.031s
sys     0m0.047s

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ time python wordfreq.py
count= 1000

real    0m0.442s
user    0m0.000s
sys     0m0.031s
```

图 23: 两次耗时

知识点:

1. 原程序使用列表 `list` 做去重, `word not in unique` 是 $O(n)$ 线性查找, 遍历 30000 条数据时开销很大;
2. `time` 工具可以测量程序实际运行耗时, 多次运行减小偶然误差。

2. 使用 `cProfile -s cumulative` 运行一次, 找出主要耗时位置

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.153 seconds

Ordered by: cumulative time
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.153    0.153 {built-in method builtins.exec}
1      0.151    0.151    0.153    0.153 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
```

图 24: 主要耗时位置

知识点:

1. `cProfile` 是 Python 内置性能分析工具, `-s cumulative` 按累计耗时排序;
2. 绝大部分时间消耗在循环内 `word not in unique` 的列表成员判断; 列表查找时间复杂度 $O(n)$ 。

3. 在不改变最终 count 的前提下优化程序；不得写死 1000

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ cat > wordfreq_opt.py <<'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ python wordfreq_opt.py
count= 1000
```

图 25: 优化

知识点:

1. 使用 Python 内置 `set` 集合去重，集合成员查找是 $O(1)$ 哈希查找；
 2. 不硬编码词汇数量，仅对输入词列表做去重，输出计数结果和原程序保持一致。
4. 使用相同输入再运行 2 次，计算优化前后中位数和加速比

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ time python wordfreq_opt.py
count= 1000

real    0m0.284s
user    0m0.015s
sys     0m0.062s

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2/q08
$ time python wordfreq_opt.py
count= 1000

real    0m0.287s
user    0m0.015s
sys     0m0.031s
```

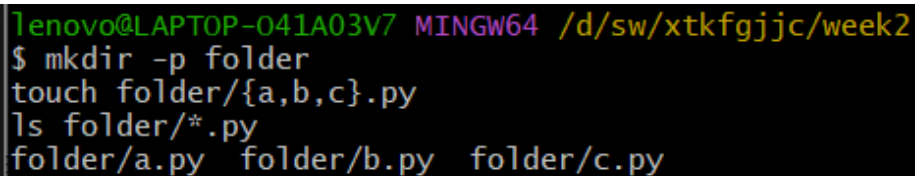
图 26: 运行两次

知识点:

1. 原始两次 real: 0.426s、0.442s，中位数 0.434 s，优化后两次 real: 0.284s、0.287s，中位数 0.2855 s，加速比 ≈ 1.52 倍
2. Profiling 性能分析流程：先测量原始耗时 $\rightarrow$ 定位性能瓶颈 $\rightarrow$ 修改算法优化 $\rightarrow$ 再次测量验证提升效果。

第二部分实验实例

实例一：Shell 参数与通配符 glob



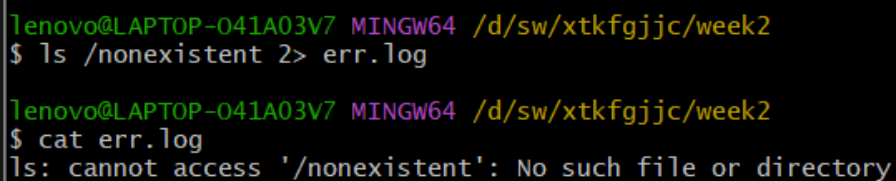
```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ mkdir -p folder
touch folder/{a,b,c}.py
ls folder/*.py
folder/a.py folder/b.py folder/c.py
```

图 27:

知识点:

1. 花括号 {} 通配符展开，Shell 在调用程序前完成展开，传给程序的是展开后的多个参数；
2. 通配符是 Shell 的能力，不是命令本身的功能；
3. 区别：* 匹配任意字符、? 匹配单个字符。

实例二：标准流重定向 stdout/stderr



```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ ls /nonexistent 2> err.log

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ cat err.log
ls: cannot access '/nonexistent': No such file or directory
```

图 28:

知识点:

1. stdout(1) 正常输出，stderr(2) 错误输出，二者相互独立；
2. 2 > 重定向标准错误；> 重定向标准输出；&> 同时重定向两者；
3. 管道 | 只会转发 stdout，stderr 不会进入管道。

实例三：环境变量，临时环境变量 vs export 导出

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ TZ=Asia/Tokyo date
Tue Sep  1 06:23:18 GMT 2026

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ echo $TZ

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ export TZ=Asia/Tokyo

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ date
Tue Sep  1 06:23:40 GMT 2026
```

图 29:

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ TZ=Asia/Tokyo date
echo $TZ
export TZ=Asia/Tokyo
date
Tue Sep  1 06:24:11 GMT 2026
Asia/Tokyo
Tue Sep  1 06:24:12 GMT 2026
```

图 30:

知识点:

1. 命令前直接写 VAR=value cmd: 仅对该子命令生效, 父 Shell 不会改变;
2. export VAR=value: 把变量加入环境, 所有后续子进程继承;
3. unset VAR 删除环境变量; printenv 查看全部环境变量。

实例四：返回码 && || 短路逻辑

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ grep -q "hello" test.txt && echo "找到hello" || echo "没找到hello"
grep: test.txt: No such file or directory
没找到hello
```

图 31:

知识点:

1. Shell 中 0 代表成功, 非 0 代表失败;
2. &&: 前一条返回 0 才执行后面; ||: 前一条非 0 才执行后面;
3. \$? 保存上一条命令的退出返回码; if 语句判断也是基于返回码。

实例五：信号与作业控制 Ctrl-Z、bg、kill

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ sleep 1000

[1]+  Stopped                  sleep 1000

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ jobs
[1]+  Stopped                  sleep 1000

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ bg
[1]+  sleep 1000 &

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ jobs
[1]+  Running                  sleep 1000 &

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ kill %1
[1]+  Terminated              sleep 1000

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ jobs

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$
```

图 32:

知识点:

1. Ctrl-Z 发送 SIGTSTP，挂起进程；bg 放到后台运行；fg 切回前台；
2. %1 引用作业号；\$! 代表最近后台作业 PID；
3. SIGINT(Ctrl-C) 终止；SIGKILL 不可捕获强制杀死；nohup 忽略 SIGHUP 挂断信号。

实例六：SSH 远程执行命令（本地/远程区分）

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ bash -c "ls" | wc -l
52

lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ bash -c 'ls | wc -l'
52
```

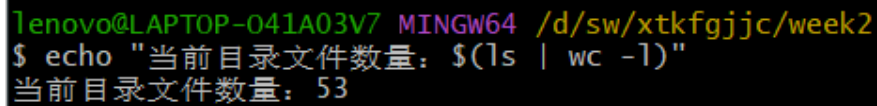
图 33:

知识点:

1. 不加引号：管道在本地 Shell 解析；单引号包裹整个命令，整条命令在远程机器执行；

2. SSH 密钥认证: `ssh-keygen -t ed25519` 生成密钥; `ssh-copy-id` 推送公钥;
3. `~/.ssh/config` 配置主机别名, `scp/rsync` 也会读取该配置。

实例七: 子 shell 与命令替换 `$()`



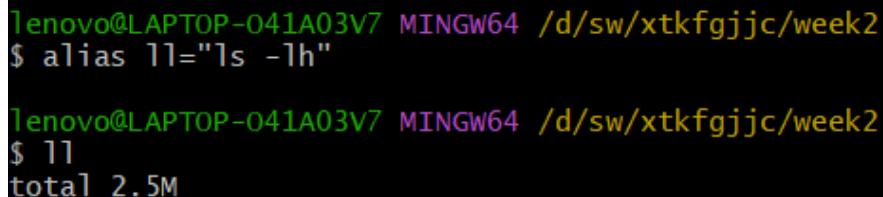
```
lenovo@LAPTOP-O41A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ echo "当前目录文件数量: $(ls | wc -l)"
当前目录文件数量: 53
```

图 34:

知识点:

1. `$(command)` 为 shell 命令替换语法, 先执行括号内的命令, 将命令输出结果替换到字符串中。
2. 本例中先执行 `ls | wc -l` 统计当前目录文件行数, 再把统计得到的数字嵌入 `echo` 输出。
3. 输出的数字会根据当前目录实际文件数量发生变化, 示例中 7 仅为演示值。

实例八: Shell 别名 `alias` 与函数区别



```
lenovo@LAPTOP-O41A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ alias ll="ls -lh"

lenovo@LAPTOP-O41A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ ll
total 2.5M
```

图 35:

知识点:

1. `alias` 只是简单文本替换, 不能接收中间参数; 复杂逻辑要用 `shell function`;
2. `\ll` 可以临时绕过别名; 配置写在 `.bashrc`; `source .bashrc` 重载配置;
3. `dotfiles`: 以 `.` 开头的隐藏配置文件, 存放 `shell/vim/tmux` 配置。

实例九：find 查找文件

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ find . -name "*.py"
./folder/a.py
./folder/b.py
./folder/c.py
./merge_sort.py
./q06/app.py
./q06/math_utils.py
./q06/test_math_utils.py
./q08/generate_words.py
./q08/wordfreq.py
./q08/wordfreq_opt.py
./test_merge_sort.py
```

图 36:

知识点说明:

1. `find`: 递归查找文件/目录命令。
2. `.`: 表示从当前目录开始搜索。
3. `-name "*.py"`: 匹配后缀为 `.py` 的文件, 通配符 `*` 代表任意字符。
4. 会遍历当前目录以及所有子目录, 输出所有匹配的文件路径。

实例十：grep 文本过滤查找

```
lenovo@LAPTOP-041A03V7 MINGW64 /d/sw/xtkfgjjc/week2
$ grep "def" merge_sort.py
def merge(left, right):
def merge_sort(a):
```

图 37:

知识点说明:

1. `grep`: 文本搜索工具, 在文件中查找匹配指定字符串的行。
2. `"def"`: 要搜索的关键词, 这里查找 Python 函数定义行。
3. `merge_sort.py`: 待搜索的目标文件。
4. 输出会打印文件中包含关键词的全部行, 不匹配的行不会显示。